游戏助手服务接口说明（BAR 服务）

目录

1 游戏助手服务接口说明（Boss 招式识别） 4

2 更新日志 4
2.1 2025-11-28 . . . 4
2.2 2025-11-21 . . . 4

3 Boss 招式识别服务 4

4 基础服务接口 4
4.1 接口 `/bar/service/enable` . . . 4
4.1.1 Method `Post` . . . 4
4.1.2 Content-Type `application/json` . . . 4
4.1.3 参数 . . . 4
4.1.4 返回 . . . 4
4.1.5 例子 . . . 5
4.2 接口 `/bar/service/init` . . . 5
4.2.1 Method `Post` . . . 5
4.2.2 Content-Type `application/json` . . . 5
4.2.3 参数 . . . 5
4.2.4 返回 . . . 6
4.2.5 例子 . . . 6
4.3 接口 `/bar/service/list` . . . 6
4.3.1 Method `Get` . . . 6
4.3.2 Content-Type `application/json` . . . 6
4.3.3 参数 . . . 6
4.3.4 返回 . . . 6
4.3.5 例子 . . . 6
4.4 接口 `/bar/service/delete` . . . 7
4.4.1 Method `Post` . . . 7
4.4.2 Content-Type `application/json` . . . 7
4.4.3 参数 . . . 7
4.4.4 返回 . . . 7
4.4.5 例子 . . . 7
4.5 接口 `/bar/boss/list` . . . 7
4.5.1 Method `Post` . . . 7
4.5.2 Content-Type `application/json` . . . 7
4.5.3 参数 . . . 7

4.5.4 返回 . . . 7
4.5.5 例子 . . . 8
4.6 接口 /bar/boss/action/list . . . 8
4.6.1 Method Post . . . 8
4.6.2 Content-Type application/json . . . 8
4.6.3 参数 . . . 8
4.6.4 返回 . . . 8
4.6.5 例子 . . . 9
4.7 接口 /bar/boss/action/delete . . . 9
4.7.1 Method Post . . . 9
4.7.2 Content-Type application/json . . . 9
4.7.3 参数 . . . 9
4.7.4 返回 . . . 9
4.7.5 例子 . . . 10
4.8 接口 /bar/boss/delete . . . 10
4.8.1 Method Post . . . 10
4.8.2 Content-Type application/json . . . 10
4.8.3 参数 . . . 10
4.8.4 返回 . . . 10
4.8.5 例子 . . . 11

5 重置接口 11
5.1 接口 /bar/service/reset . . . 11
5.1.1 Method Post . . . 11
5.1.2 Content-Type application/json . . . 11
5.1.3 参数 . . . 11
5.1.4 返回 . . . 11
5.1.5 例子 . . . 12

6 标注接口 12
6.1 接口 /bar/boss/action/insert . . . 12
6.1.1 Method Post . . . 12
6.1.2 Content-Type multipart/form-data . . . 12
6.1.3 参数 . . . 12
6.1.4 返回 . . . 13
6.1.5 例子 . . . 13
6.2 接口 /bar/annotation/add . . . 13
6.2.1 Method Post . . . 14
6.2.2 Content-Type application/json . . . 14
6.2.3 参数 . . . 14
6.2.4 返回 . . . 14
6.2.5 例子 . . . 14
6.3 接口 /bar/annotation/remove . . . 15
6.3.1 Method Post . . . 15
6.3.2 Content-Type application/json . . . 15

6.3.3 参数 . . . 15
6.3.4 返回 . . . 15
6.3.5 例子 . . . 15
6.4 接口 `/bar/annotation/all/remove` . . . 16
6.4.1 Method `Post` . . . 16
6.4.2 Content-Type `application/json` . . . 16
6.4.3 参数 . . . 16
6.4.4 返回 . . . 16
6.4.5 例子 . . . 16
6.5 接口 `/bar/service/process` . . . 16
6.5.1 Method `Post` . . . 16
6.5.2 Content-Type `application/json` . . . 16
6.5.3 参数 . . . 16
6.5.4 返回 . . . 16
6.5.5 例子 . . . 17
6.6 接口 `/bar/frame/track` . . . 17
6.6.1 Method `Post` . . . 17
6.6.2 Content-Type `application/json` . . . 17
6.6.3 参数 . . . 17
6.6.4 返回 . . . 17
6.6.5 例子 . . . 17
6.7 接口 `/bar/service/build` . . . 18
6.7.1 Method `Post` . . . 18
6.7.2 Content-Type `application/json` . . . 18
6.7.3 参数 . . . 18
6.7.4 返回 . . . 18
6.7.5 例子 . . . 18

7 识别和跟踪接口 18
7.1 接口 `/bar/boss/query` . . . 18
7.1.1 Method `Post` . . . 19
7.1.2 Content-Type `multipart/form-data` . . . 19
7.1.3 参数 . . . 19
7.1.4 返回 . . . 19
7.1.5 例子 . . . 19

8 实时标注和跟踪接口 20
8.1 接口 `/bar/annotation/live/add` . . . 21
8.1.1 Method `Post` . . . 21
8.1.2 Content-Type `multipart/form-data` . . . 21
8.1.3 参数 . . . 21
8.1.4 返回 . . . 21
8.1.5 例子 . . . 21
8.2 接口 `/bar/frame/live/track` . . . 22
8.2.1 Method `Post` . . . 22

8.2.2 Content-Type `multipart/form-data` 22
8.2.3 参数 22
8.2.4 返回 22
8.2.5 例子 22

1 游戏助手服务接口说明（Boss 招式识别）

编辑日期 `2025-11-28` 。
游戏助手服务版本 `2025-11-28` 。

2 更新日志

2.1 2025-11-28

- 增加 `重置接口` 中的 `reset_database` 参数的详细说明。
- 增加各个大类中 `重置` 时是否需要清空数据库的补充说明。

2.2 2025-11-21

- 初版。
- 提供 `Boss 招式识别服务` 接口说明。

3 Boss 招式识别服务

该服务分为四大类。

1. 基础服务接口
2. 标注接口
3. 识别和跟踪接口
4. 实时标注和跟踪接口

注意：除 `基础服务接口` 外，其它三大类的接口都有严格的调用顺序，并且在首次调用某一大类的接口前，必须先调用 `重置接口` ，否则无法正确工作。
注意：同一时间只能有一个接口在调用，不支持并行操作。

4 基础服务接口

基础服务接口主要是服务的启停、Boss 实例的增删查，不涉及标注、识别和跟踪。

4.1 接口 `/bar/service/enable`

启停或查询 Boss 招式识别服务。

4.1.1 Method `Post`

4.1.2 Content-Type `application/json`

4.1.3 参数

1. `enable` ：Bool 类型。`null` 表示查询，`true` 启用，`false` 停用。

4.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "bar": bool 类型。是否启用 Boss 招式识别服务。
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。 `enable` 为 `null` 时返回 `data` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

4.1.5 例子

```
# 查询 Boss 招式识别服务是否启用

import json
import requests

url = f"http://127.0.0.1:9190/bar/service/enable"
data = { "enable": None }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
  "code": "ok",
  "data": {
    "bar": true
  }
}
"""
```

```
# 启用 Boss 招式识别服务

import json
import requests

url = f"http://127.0.0.1:9190/bar/service/enable"
data = { "enable": True }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
  "code": "ok"
}
"""
```

4.2 接口 `/bar/service/init`

创建 Boss 招式识别服务实例。

4.2.1 Method `Post`

4.2.2 Content-Type `application/json`

4.2.3 参数

1. `instance_id` ：String 类型。实例 ID。

4.2.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

4.2.5 例子

```
# 创建实例，实例 ID 为`test_instance`

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/bar/service/init"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

4.3 接口 `/bar/service/list`

列出所有实例。

4.3.1 Method `Get`

4.3.2 Content-Type `application/json`

4.3.3 参数

无

4.3.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "instances_id": [
            " 实例 ID 列表"
        ]
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

4.3.5 例子

```
# 列出所有实例

import requests
```

```
url = f"http://127.0.0.1:9190/bar/service/list"
resp = requests.get(url)
resp_data = resp.json()
# resp_data: {"code": "ok", "data": {"instances_id": ["test_instance"]}}
```

4.4 接口 /bar/service/delete

删除实例。

4.4.1 Method Post

4.4.2 Content-Type application/json

4.4.3 参数

1. instance_id : String 类型。实例 ID。

4.4.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code :
1. ok : 成功。不会出现 msg 。
2. fail : 失败。会有 msg 给出失败原因。

4.4.5 例子

```
# 删除实例`test_instance`

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/bar/service/delete"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

4.5 接口 /bar/boss/list

列出指定实例下的所有 Boss。

4.5.1 Method Post

4.5.2 Content-Type application/json

4.5.3 参数

1. instance_id : String 类型。实例 ID。

4.5.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
```

```
        "boss_ids": [
            "Boss ID 列表"
        ]
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

4.5.5 例子

```
# 列出所有 Boss,
# 实例 ID 为`test_instance`。

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/bar/boss/list"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
  "code": "ok",
  "data": {
    "boss_ids": [
      "test_boss"
    ]
  }
}
"""
```

4.6 接口 `/bar/boss/action/list`

列出指定 Boss 中的招式 ID。

4.6.1 Method `Post`

4.6.2 Content-Type `application/json`

4.6.3 参数

1. `instance_id` ：String 类型。实例 ID。
2. `boss_id` ：String 类型。Boss ID。

4.6.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "action_ids": [
            " 招式 ID 列表"
        ]
```

```
    }
}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

4.6.5 例子

```
# 列出招式 ID,
# 实例 ID 为`test_instance`,
# Boss ID 为`test_boss`

import json
import requests

instance_id = "test_instance"
boss_id = "test_boss"
url = f"http://127.0.0.1:9190/bar/boss/action/list"
data = { "instance_id": instance_id, "boss_id": boss_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
  "code": "ok",
  "data": {
    "action_ids": [
      "test_action"
    ]
  }
}
"""
```

4.7 接口 `/bar/boss/action/delete`

从指定的 Boss 中删除招式。

4.7.1 Method `Post`

4.7.2 Content-Type `application/json`

4.7.3 参数

1. `instance_id` ：String 类型。实例 ID。
2. `boss_id` ：String 类型。Boss ID。
3. `action_ids` ：List[String] 类型。招式 ID 列表。

4.7.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "deleted": [
            " 成功删除的招式 ID 列表"
        ],
```

```
        "not_exist": [
            " 不存在的招式 ID 列表"
        ]
    }
}
```

返回的 `code` :
1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。不会出现 `data` 。

4.7.5 例子

```
# 删除 Boss 招式,
# 实例 ID 为`test_instance`,
# Boss ID 为`test_boss`,
# 招式 ID 列表为`test_action`,
# 以及不存在的招式 ID 为`not_exists_id_1`和`not_exists_id_2`。

import json
import requests

instance_id = "test_instance"
boss_id = "test_boss"
url = f"http://127.0.0.1:9190/bar/boss/action/delete"
data = {
    "instance_id": instance_id,
    "boss_id": boss_id,
    "action_ids": [ "test_action", "not_exists_id_1", "not_exists_id_2" ]
}
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
""" resp_data
{
  "code": "ok",
  "data": {
    "deleted": [ "test_action" ],
    "not_exist": [ "not_exists_id_1", "not_exists_id_2" ]
  }
}
"""
```

4.8 接口 `/bar/boss/delete`

从指定实例删除一个 Boss。

4.8.1 Method `Post`

4.8.2 Content-Type `application/json`

4.8.3 参数

1. `instance_id` : String 类型。实例 ID。
2. `boss_id` : String 类型。Boss ID。

4.8.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

4.8.5 例子

```
# 删除 Boss，
# 实例 ID 为`test_instance`，
# Boss ID 为`test_boss`

import json
import requests

instance_id = "test_instance"
boss_id = "test_boss"
url = f"http://127.0.0.1:9190/bar/boss/delete"
data = { "instance_id": instance_id, "boss_id": boss_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

5 重置接口

在首次调用某一大类的接口前，必须先调用 `重置接口` ，否则无法正确工作。
具体调用细则参照各大类的调用说明。

5.1 接口 `/bar/service/reset`

重置 Boss 招式识别服务的内部状态，但不会清空已完成的标注内容。

5.1.1 Method `Post`

5.1.2 Content-Type `application/json`

5.1.3 参数

1. `instance_id` ：String 类型。实例 ID。

2. `reset_database` ：Bool 类型。是否清空并重置 Boss 招式数据库，默认 False。
   注意：在切换实例时，请传 **True**。确保新实例不会受旧实例数据库的影响。
   注意：标注阶段，请传 **False**，避免先前的标注内容被清空。
   注意：从标注阶段切换至识别和跟踪阶段时，请传 **True**，会重新加载已保存在磁盘上的数据库。
   注意：从标注阶段切换至实时标注和跟踪时，请传 **True**，确保实时标注和跟踪是基于空的数据库进行的。

5.1.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

5.1.5 例子

```
# 重置服务状态，实例 ID 为`test_instance`。

import json
import requests

instance_id = "test_instance"
url = f"http://127.0.0.1:9190/bar/service/reset"
data = { "instance_id": instance_id, "reset_database": False }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6 标注接口

标注的流程如下，请务必严格按照顺序调用。

1. 重置：在插入前必须重置服务，根据需求决定是否清空数据库。
2. 插入：插入招式图片。
3. 标注：在图片上标上点或框。
4. 处理：对所有图片进行分析计算。
5. 跟踪：对图片中的 Boss 招式进行分割，并返回 mask。
6. 构建：与数据库中已有的 Boss 招式合并，然后保存至磁盘。

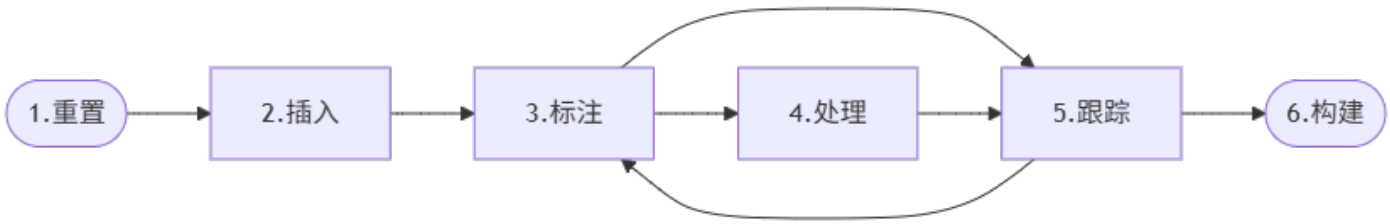


Figure 1: 标注流程

其中，在第 2 步插入完成后，可以 3->4->5->3 的顺序循环执行。
第 3 步标注完当前图片后，可以立即对当前图片进行第 5 步跟踪查看效果，但是不能对其它未标注过的图片进行跟踪。若跟踪效果不满意，可以继续标注、跟踪，直至结果满意。
第 4 步处理完后，可以对所有图片进行跟踪，若发现有效果不满意的图片，可以回到第 3 步开始标注。
只有在最后一步 `构建` 后，标注的内容才会存入数据库。除此之外，可以从任意一步放弃从第 1 步 `重置` 开始。

6.1 接口 `/bar/boss/action/insert`

插入 Boss 招式的图片，每张图片的分辨率必须一样，不能有大有小。
如果插入的 Boss ID 和招式 ID 已经存在，之前的图片全部会被删除，替换成当前插入的图片。

6.1.1 Method `Post`

6.1.2 Content-Type `multipart/form-data`

6.1.3 参数

1. `instance_id` : String 类型。实例 ID。

2. `boss_id` ：String 类型。Boss ID。
3. `action_id` ：String 类型。招式 ID。
4. `frames` ：List[Bytes] 类型。图片列表，列表中的每个元素是文件的 Byte 数据。
5. `is_background_action` ：Bool 类型。默认 False。是否为背景招式。

关于背景招式：此处的 `背景` 不是字面上的意思，指的是一种特殊的招式分类，通常是 Boss 空闲状态的样子，或者被定身时的样子。当 Boss 没有在释放招式时，会被识别为背景招式，能一定程度提高特定招式的识别率，也能更方便的忽略掉不需要的图像帧。

6.1.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` ：
1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。

6.1.5 例子

```python
# 插入 Boss 招式图片，
# 实例 ID 为`test_instance`，
# Boss ID 为`test_boss`，
# 招式 ID 为`test_action`。

import json
import requests

pic_1 = r"C:\Users\Public\Pictures\0001.jpg"
pic_2 = r"C:\Users\Public\Pictures\0002.jpg"

instance_id = "test_instance"
boss_id = "test_boss"
action_id = "test_action"
url = f"http://127.0.0.1:9190/bar/boss/action/insert"
data = {
    "instance_id": instance_id,
    "boss_id": boss_id,
    "action_id": action_id,
    "is_background_action": False
}
files = [ ("frames", open(pic_1, "rb")), ("frames", open(pic_2, "rb")) ]
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.2 接口 `/bar/annotation/add`

对指定的图片添加标注点，每次添加都是增量的，不会清除之前添加的点。
若要全新添加，请先调用 `/bar/annotation/remove` 接口清空指定图片的所有标注点。

6.2.1 Method `Post`

6.2.2 Content-Type `application/json`

6.2.3 参数

1. `instance_id` ：String 类型。实例 ID。
2. `boss_id` ：String 类型。Boss ID。
3. `frame_index` ：Int 类型。图片索引，从 0 开始，与接口 `/bar/boss/action/insert` 中的 `frames` 列表对应。
4. `points` ：List[List[Int]] 类型。标注点列表，格式为 [[x1, y1], [x2, y2], [xN, yN]]，每个点是以图片左上角为原点的 [x, y] 坐标点。
5. `labels` ：List[Int] 类型。标注点类别，与标注点一一对应，格式为 [1, 0, 1]，`0` 表示该标注点不想被识别跟踪，`1` 表示该标注点希望被识别跟踪的。
6. `box` ：List[List[Int]] 类型。标注矩形框，表示想被识别跟踪的区域，仅支持一个。格式为 [[x1, y1, x2, y2]]，以图片左上角为原点，x1 和 y1 是矩形框左上角的点，x2 和 y2 是矩形框右下角的点。

注意：`points` 和 `box` 可以同时传，但至少传其中一个，不能都不传。

6.2.4 返回

返回有 2 种，成功的情况下会返回 mask 的 png 格式的文件数据。
失败的情况下会返回 json，格式如下：

```
{
    "code": "fail",
    "msg": " 失败原因"
}
```

6.2.5 例子

```
# 添加标注点，
# 实例 ID 为`test_instance`，
# Boss ID 为`test_boss`，
# 图片索引为`0`，
# 标注点有 5 个，
# 其中前三个是需要跟踪的，
# 后两个不需要跟踪，
# 不传 Box。

import json
import requests

instance_id = "test_instance"
boss_id = "test_boss"
frame_index = 0
points = [  [674, 236], [678, 324], [682, 384],[492, 446], [485, 569] ]
labels = [ 1, 1, 1, 0, 0 ]
box = None # [[567, 199, 835, 498]]

url = f"http://127.0.0.1:9190/bar/annotation/add"
data = {
    "instance_id": instance_id,
    "boss_id": boss_id,
    "frame_index": frame_index,
    "points": points,
```

```
    "labels": labels,
    "box": box
}
resp = requests.post(url, data=json.dumps(data))
resp_headers = resp.headers
resp_content_type = resp_headers.get('content-type')
if resp_content_type == 'application/json':
    content = resp.json() # { "code": "fail", "msg": " 失败原因" }
else:
    content = resp.content # mask png file bytes
```

6.3 接口 /bar/annotation/remove

移除指定图片的标注点。

6.3.1 Method Post

6.3.2 Content-Type application/json

6.3.3 参数

1. instance_id ：String 类型。实例 ID。
2. boss_id ：String 类型。Boss ID。
3. frame_index ：Int 类型。图片索引，从 0 开始，与接口 /bar/boss/action/insert 中的 frames 列表对应。

6.3.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：

1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

6.3.5 例子

```
# 移除指定图片的标注点，
# 实例 ID 为`test_instance`，
# Boss ID 为`test_boss`，
# 图片索引为`0`。

import json
import requests

instance_id = "test_instance"
boss_id = "test_boss"
frame_index = 0

url = f"http://127.0.0.1:9190/bar/annotation/remove"
data = { "instance_id": instance_id, "boss_id": boss_id, "frame_index": frame_index }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.4 接口 `/bar/annotation/all/remove`

移除所有图片的标注点。

6.4.1 Method `Post`

6.4.2 Content-Type `application/json`

6.4.3 参数

1. `instance_id` : String 类型。实例 ID。

6.4.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` :

1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。

6.4.5 例子

```
# 移除所有图片的标注点，
# 实例 ID 为`test_instance`。

import json
import requests

instance_id = "test_instance"

url = f"http://127.0.0.1:9190/bar/annotation/all/remove"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.5 接口 `/bar/service/process`

处理所有图片，处理完成后即使没有标注过的图片也能跟踪。

6.5.1 Method `Post`

6.5.2 Content-Type `application/json`

6.5.3 参数

1. `instance_id` : String 类型。实例 ID。

6.5.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 `code` :

1. `ok` : 成功。不会出现 `msg` 。
2. `fail` : 失败。会有 `msg` 给出失败原因。

6.5.5 例子

```
# 处理所有图片，
# 实例 ID 为`test_instance`。

import json
import requests

instance_id = "test_instance"

url = f"http://127.0.0.1:9190/bar/service/process"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

6.6 接口 /bar/frame/track

跟踪指定的图片，未标注过的图片无法跟踪，只有在调用接口 `/bar/service/process` 后才能跟踪所有图片。

6.6.1 Method `Post`

6.6.2 Content-Type `application/json`

6.6.3 参数

1. `instance_id` ：String 类型。实例 ID。
2. `boss_id` ：String 类型。Boss ID。
3. `frame_index` ：Int 类型。图片索引，从 0 开始，与接口 `/bar/boss/action/insert` 中的 `frames` 列表对应。

6.6.4 返回

返回有 2 种，成功的情况下会返回 mask 的 png 格式的文件数据。
失败的情况下会返回 json，格式如下：

```
{
    "code": "fail",
    "msg": " 失败原因"
}
```

6.6.5 例子

```
# 跟踪指定图片，
# 实例 ID 为`test_instance`，
# Boss ID 为`test_boss`，
# 图片索引为`0`。

import json
import requests

instance_id = "test_instance"
boss_id = "test_boss"
frame_index = 0

url = f"http://127.0.0.1:9190/bar/frame/track"
data = { "instance_id": instance_id, "boss_id": boss_id, "frame_index": frame_index }
resp = requests.post(url, data=json.dumps(data))
```

```
resp_headers = resp.headers
resp_content_type = resp_headers.get('content-type')
if resp_content_type == 'application/json':
    content = resp.json() # { "code": "fail", "msg": " 失败原因" }
else:
    content = resp.content # mask png file bytes
```

6.7 接口 /bar/service/build

构建并保存数据库，调用此接口意味着当前标注流程的结束。

6.7.1 Method Post

6.7.2 Content-Type application/json

6.7.3 参数

1. instance_id ：String 类型。实例 ID。

6.7.4 返回

格式：

```
{"code": " 返回代码", "msg": " 失败原因"}
```

返回的 code ：

1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。

6.7.5 例子

```
# 构建并保存数据库，
# 实例 ID 为`test_instance`。

import json
import requests

instance_id = "test_instance"

url = f"http://127.0.0.1:9190/bar/service/build"
data = { "instance_id": instance_id }
resp = requests.post(url, data=json.dumps(data))
resp_data = resp.json()
# resp_data: {"code": "ok"}
```

7 识别和跟踪接口

识别和跟踪就一个接口，但是在首次调用前必须调用 重置接口 。

1. 重置：在识别和跟踪前必须重置服务，包括清空数据库。
2. 识别和跟踪：识别和跟踪传入的图片。

7.1 接口 /bar/boss/query

识别和跟踪 Boss，返回识别的结果和 mask。

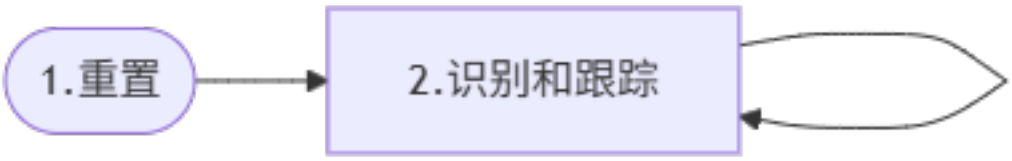


Figure 2: 识别和跟踪流程

7.1.1 Method `Post`

7.1.2 Content-Type `multipart/form-data`

7.1.3 参数

1. `instance_id` ：String 类型。实例 ID。
2. `boss_id` ：String 或 List[String] 类型。Boss ID 或包含多个 Boss ID 的列表。如果传空字符串，将自动识别图片中的 Boss。
3. `frame` ：Bytes 类型。图片文件的 Byte 数据。
4. `k_actions` ：Int 类型。默认是 3。返回识别到的每个 Boss 的招式个数。

7.1.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "results": [
            {
                "boss_id": "Boss ID",
                "action_id": " 招式 ID",
                "score": float 类型，识别分数，范围[0, 1],
                "frame_hits": int 类型，命中该图片的次数。
            }
        ],
        "masks": [
            {
                "boss_id": "Boss ID",
                "mask": " 经 base64 编码的 mask 的 png 数据"
            }
        ]
    }
}
```

返回的 `code` ：

1. `ok` ：成功。不会出现 `msg` 。
2. `fail` ：失败。会有 `msg` 给出失败原因。不会出现 `data` 。

7.1.5 例子

```
# 识别和跟踪 Boss，
# 实例 ID 为`test_instance`，
# Boss ID 为`test_boss`，
# 返回的招式个数为 3 个。

import json
```

```python
import requests

instance_id = "test_instance"
boss_id = "" # [ "test_boss" ]
frame = r"C:\Users\Public\Pictures\0003.jpg"
k_actions = 3

url = f"http://127.0.0.1:9190/bar/boss/query"
data = { "instance_id": instance_id, "boss_id": boss_id, "k_actions": k_actions }
files = [ ("frame", open(frame, "rb")) ]
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "results": [
            {
                "boss_id": "test_boss",
                "action_id": "test_action",
                "score": 0.7243980566660563,
                "frame_hits": 36
            }
        ],
        "masks": [
            {
                "boss_id": "test_boss",
                "mask": "iVBORw0KGg ... lFTkSuQmCC"
            }
        ]
    }
}
"""
```

8 实时标注和跟踪接口

实时标注和跟踪类似于之前的标注和识别跟踪，只是不会生成数据库，重置后就会失效，需要重新标注。

1. 重置：在实时标注前必须重置服务，包括清空数据库。
2. 实时标注：在单张图片上标上点或框。
3. 实时跟踪：对图片中的对象进行分割，并返回 mask。

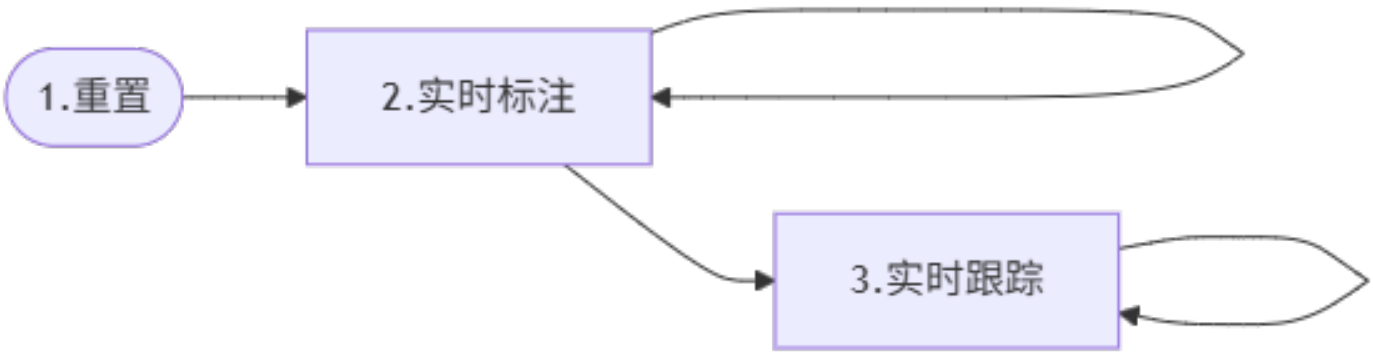


Figure 3: 实时标注和跟踪流程

注意：首次实时标注前需要先重置服务，包括清空数据库，在实时跟踪前，可以反复进行实时标注，一旦进入实时跟踪，若要再去实时标注，需要先重置服务，包括清空数据库。

8.1 接口 /bar/annotation/live/add

实时标注目标，返回识别的结果和 mask。
实时跟踪前，可以反复进行实时标注。

8.1.1 Method Post

8.1.2 Content-Type multipart/form-data

8.1.3 参数

1. instance_id ：String 类型。实例 ID。
2. boss_id ：String 类型。Boss ID。
3. frame ：Bytes 类型。图片文件的 Byte 数据。
4. points ：List[List[Int]] 类型。标注点列表，格式为 [[x1, y1], [x2, y2], [xN, yN]]，每个点是以图片左上角为原点的 [x, y] 坐标点。
5. labels ：List[Int] 类型。标注点类别，与标注点一一对应，格式为 [1, 0, 1]，0 表示该标注点不想被识别跟踪，1 表示该标注点希望被识别跟踪的。
6. box ：List[List[Int]] 类型。标注矩形框，表示想被识别跟踪的区域，仅支持一个。格式为 [[x1, y1, x2, y2]]，以图片左上角为原点，x1 和 y1 是矩形框左上角的点，x2 和 y2 是矩形框右下角的点。

注意：points 和 box 可以同时传，但至少传其中一个，不能都不传。

8.1.4 返回

返回有 2 种，成功的情况下会返回 mask 的 png 格式的文件数据。
失败的情况下会返回 json，格式如下：

```
{
    "code": "fail",
    "msg": " 失败原因"
}
```

8.1.5 例子

```
# 添加实时标注点，
# 实例 ID 为`test_instance`，
# Boss ID 为`test_live_boss`，
# 标注点有 5 个，
# 其中前三个是需要跟踪的，
# 后两个不需要跟踪，
# 不传 Box。

import json
import requests

instance_id = "test_instance"
boss_id = "test_live_boss"
frame = r"C:\Users\Public\Pictures\0001.jpg"
points = [  [674, 236], [678, 324], [682, 384],[492, 446], [485, 569] ]
labels = [ 1, 1, 1, 0, 0 ]
box = None # [[567, 199, 835, 498]]
```

```
url = f"http://127.0.0.1:9190/bar/annotation/live/add"
data = {
    "instance_id": instance_id,
    "boss_id": boss_id,
    "points": points,
    "labels": labels,
    "box": box
}
files = [ ("frame", open(frame, "rb")) ]
resp = requests.post(url, data=data, files=files)
resp_headers = resp.headers
resp_content_type = resp_headers.get('content-type')
if resp_content_type == 'application/json':
    content = resp.json() # { "code": "fail", "msg": " 失败原因" }
else:
    content = resp.content # mask png file bytes
```

8.2 接口 /bar/frame/live/track

实时跟踪目标，返回识别的结果和 mask。

8.2.1 Method Post

8.2.2 Content-Type multipart/form-data

8.2.3 参数

1. instance_id ：String 类型。实例 ID。
2. frame ：Bytes 类型。图片文件的 Byte 数据。

8.2.4 返回

格式：

```
{
    "code": " 返回代码",
    "msg": " 失败原因",
    "data": {
        "masks": [
            {
                "boss_id": "Boss ID",
                "mask": " 经 base64 编码的 mask 的 png 数据"
            }
        ]
    }
}
```

返回的 code ：
1. ok ：成功。不会出现 msg 。
2. fail ：失败。会有 msg 给出失败原因。不会出现 data 。

8.2.5 例子

```
# 实时跟踪目标，
# 实例 ID 为`test_instance`。

import json
import requests
```

```
instance_id = "test_instance"
frame = r"C:\Users\Public\Pictures\0003.jpg"

url = f"http://127.0.0.1:9190/bar/frame/live/track"
data = { "instance_id": instance_id }
files = [ ("frame", open(frame, "rb")) ]
resp = requests.post(url, data=data, files=files)
resp_data = resp.json()
""" resp_data
{
    "code": "ok",
    "data": {
        "masks": [
            {
                "boss_id": "test_live_boss",
                "mask": "iVBORw0KGg ... lFTkSuQmCC"
            }
        ]
    }
}
"""
```